33.2 核心命令解析

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~62k tokens，输出 ~7k tokens（本节）

OpenClaw CLI 注册了超过 25 个顶层子命令，涵盖 Gateway 管理、消息收发、模型配置、安全审计等方方面面。本节按使用频率和功能分类，逐一解析每个核心命令的设计与实现。

33.2.1 `gateway` — 启动/管理 Gateway 服务器

`gateway` 是 OpenClaw 最核心的命令——它负责启动、停止和管理 WebSocket Gateway 服务器。

子命令一览

子命令	说明
`gateway` （默认）	前台运行 Gateway 服务器
`gateway run`	同上，显式前台运行
`gateway status`	显示服务状态 + 探测 Gateway
`gateway install`	安装系统服务（launchd/systemd/schtasks）
`gateway uninstall`	卸载系统服务
`gateway start`	启动系统服务
`gateway stop`	停止系统服务
`gateway restart`	重启系统服务
`gateway call <method>`	调用 Gateway RPC 方法
`gateway health`	获取 Gateway 健康状态
`gateway probe`	综合探测（可达性 + 发现 + 健康）
`gateway discover`	通过 Bonjour 发现局域网内的 Gateway

子命令	说明
`gateway usage-cost`	查询使用成本统计

服务生命周期管理

`gateway install/uninstall/start/stop/restart` 五个命令委托给 `daemon-cli.ts` 实现，根据操作系统选择不同的服务管理后端：

- **macOS** — launchd（`launchctl`）
- **Linux** — systemd（`systemctl`）
- **Windows** — schtasks（计划任务）

```
# 安装为系统服务，指定端口和运行时
openclaw gateway install --port 18789 --runtime node

# 带 --force 覆盖已有服务配置
openclaw gateway install --force

# 查看服务状态 + 探测运行中的 Gateway
openclaw gateway status --deep
```

Gateway 发现（Bonjour）

`gateway discover` 用 mDNS/Bonjour 协议在局域网内搜索运行中的 Gateway 实例：

```
// src/cli/gateway-cli/register.ts（简化）
const beacons = await discoverGatewayBeacons({ timeoutMs, wideAreaDomain });
const deduped = dedupeBeacons(beacons); // 去重
```

> **衍生解释 — Bonjour/mDNS**
>
> Bonjour（也称 mDNS/DNS-SD）是 Apple 开发的零配置网络协议，允许设备在局域网内自动发现服务，无需 DNS 服务器。当 Gateway 启动时，它会向局域网广播自己的存在（包括 IP 地址和端口），其他设备可以通过 Bonjour 找到它。这使得多台设备可以在家庭/办公网络中自动发现并连接到 Gateway。

Gateway RPC 调用

`gateway call` 提供了一个通用的 RPC 调用接口，允许直接调用 Gateway 暴露的方法：

```
# 调用 health 方法
openclaw gateway call health --json

# 调用 cron.list 并传递参数
openclaw gateway call cron.list --params '{"active": true}'
```

33.2.2 `agent` — 发送消息给 AI Agent

`agent` 命令是用户与 AI Agent 直接对话的主要方式，它通过 Gateway WebSocket 连接发送消息并接收回复。

```
# 基本用法：向指定号码开始新会话
openclaw agent --to +15555550123 --message "分析今天的日志"

# 指定 Agent 和思考级别
openclaw agent --agent ops --message "总结报告" --thinking medium

# 继续已有会话
openclaw agent --session-id 1234 --message "继续上面的分析"

# 让 Agent 回复并通过 Slack 发送
openclaw agent --message "生成周报" --deliver --reply-channel slack --reply-to "#reports'
```

关键选项

选项	说明
`--message <text>`	**必填**，消息内容
`--to <number>`	接收者号码（E.164 格式），用于派生会话 key
`--session-id <id>`	直接指定会话 ID
`--agent <id>`	指定 Agent（覆盖路由绑定）
`--thinking <level>`	思考级别：off / minimal / low / medium / high
`--channel <channel>`	投递通道：last / whatsapp / telegram / discord / ...
`--deliver`	将 Agent 回复发送到选定通道
`--local`	在本地运行嵌入式 Agent（需要 API key）

选项	说明
`--json`	以 JSON 格式输出结果
`--timeout <seconds>`	超时时间（默认 600 秒）

agents 管理子命令

除了 `agent` （单数），还有 `agents` （复数）命令用于管理多 Agent 配置：

```
openclaw agents list --bindings    # 列出所有 Agent 及路由绑定
openclaw agents add ops --workspace ~/ops --model gpt-4  # 添加新 Agent
openclaw agents set-identity --agent main --from-identity  # 从 IDENTITY.md 设置身份
openclaw agents delete ops --force   # 删除 Agent
```

33.2.3 `send` — 通过通道发送消息

`send` 命令（实际注册为 `message send` ）允许直接通过指定通道发送消息，绕过 Agent 层：

```
openclaw message send --target +15555550123 --message "Hi"
openclaw message send --channel telegram --target @mychat --message "Hello"
openclaw message send --target +15555550123 --message "看这张图" --media photo.jpg
```

`message` 命令组还包括丰富的子命令：

子命令	说明
`message send`	发送文本/媒体消息
`message broadcast`	群发消息
`message poll`	创建投票
`message react` / `unreact`	添加/移除表情反应
`message read` / `edit` / `delete`	读取/编辑/删除消息
`message pin` / `unpin`	置顶/取消置顶
`message permissions`	权限管理
`message search`	搜索消息

子命令	说明
`message thread`	线程消息操作
`message emoji / sticker`	Emoji 和贴纸管理

每个子命令的注册被拆分到独立的模块文件中（如 `register.send.ts` 、 `register.poll.ts` ），通过 `createMessageCliHelpers` 共享通道选项和公共逻辑。

33.2.4 `channels` — 通道管理（登录/配置）

`channels` 命令管理聊天通道的账户配置和认证：

```
openclaw channels list                      # 列出所有通道 + 认证状态
openclaw channels login --channel whatsapp # 链接 WhatsApp Web（显示 QR 码）
openclaw channels add --channel telegram --token <bot-token>  # 添加 Telegram Bot
openclaw channels status --probe            # 探测通道凭证状态
openclaw channels capabilities              # 显示通道能力（权限/功能）
openclaw channels resolve @user --channel discord  # 解析用户名到 ID
```

子命令详解

子命令	说明
`channels list`	列出所有已配置通道及认证 Profile
`channels status`	显示 Gateway 通道连接状态
`channels capabilities`	展示通道的权限/功能矩阵
`channels resolve`	将名称解析为通道 ID
`channels logs`	查看最近的通道日志
`channels add`	添加或更新通道账户
`channels remove`	禁用或删除通道账户
`channels login`	链接通道账户（如 WhatsApp QR 码扫描）
`channels logout`	登出通道会话

`channels add` 命令支持超过 30 个选项，涵盖所有支持的通道类型（WhatsApp、Telegram、Discord、Slack、Signal、iMessage、Matrix、Tlon 等），每种通道有各自特定的认证参数。

33.2.5 `models` — 模型管理（列表/设置/扫描）

`models` 命令管理 AI 模型的配置、认证和回退策略：

```
openclaw models                        # 显示当前模型状态
openclaw models list --all             # 列出完整模型目录
openclaw models set gpt-4o             # 设置默认模型
openclaw models scan --set-default     # 扫描免费模型并设为默认
openclaw models status --probe         # 探测认证状态
```

命令树

```
models
├── list                  列出模型（支持 --all/--local/--provider 过滤）
├── status                显示配置的模型状态（支持 --probe 在线探测）
├── set <model>           设置默认文本模型
├── set-image <model>     设置默认图像模型
├── scan                  扫描 OpenRouter 免费模型
├── auth                  认证管理
│   ├── add               交互式认证助手
│   ├── login             Provider 插件认证流（OAuth/API key）
│   ├── setup-token       运行 Provider CLI 创建 Token
│   ├── paste-token       粘贴 Token 到 auth-profiles.json
│   ├── login-github-copilot  GitHub Copilot 设备认证流
│   └── order             Per-Agent 认证顺序管理
│       ├── get           查看认证顺序
│       ├── set           设置认证顺序
│       └── clear         清除认证顺序覆盖
├── aliases               模型别名管理
│   ├── list              列出别名
│   ├── add <alias> <model>   添加别名
│   └── remove <alias>        移除别名
├── fallbacks             文本模型回退链
│   ├── list / add / remove / clear
└── image-fallbacks       图像模型回退链
    ├── list / add / remove / clear
```

`models` 是 CLI 中命令树最深的命令之一，涉及三级嵌套（`models auth order set`）。

33.2.6 `sessions` — 会话管理

```
openclaw sessions                       # 列出所有会话
openclaw sessions --active 120          # 仅显示最近 2 小时活跃的会话
openclaw sessions --json                # JSON 格式输出
openclaw sessions --store ./sessions    # 指定会话存储路径
```

`sessions` 命令是快速路由（Fast-Path）支持的命令之一，可以绕过完整的 Commander.js 管道直接执行。它显示每个会话的接收者、最后活跃时间、Token 使用量等信息。

33.2.7 `skills` — 技能管理

```
openclaw skills                         # 列出所有技能
openclaw skills list --eligible         # 仅显示可用技能
openclaw skills info browser            # 查看技能详细信息
openclaw skills check                   # 检查技能就绪状态
```

`skills` 命令的输出格式经过精心设计——用表格渲染（`renderTable`），每个技能显示状态图标（✓ ready / ⏸ disabled / 🚫 blocked / ✗ missing）、Emoji 名称、描述和来源。`skills info` 还会显示详细的需求检查（二进制依赖、环境变量、配置项、操作系统要求），并提供安装建议。

末尾会附带一条 ClawHub 提示：`Tip: use npx clawhub to search, install, and sync skills.`

33.2.8 `browser` — 浏览器控制

`browser` 命令管理 OpenClaw 的专用浏览器实例（Chrome/Chromium），支持远程操控：

```
openclaw browser status                 # 浏览器状态
openclaw browser open https://example.com  # 打开 URL
openclaw browser screenshot             # 截图
openclaw browser click --selector "#btn"   # 点击元素
```

命令按功能分为七个模块：

模块	说明
`browser-cli-manage`	生命周期管理（status/open/close）
`browser-cli-extension`	浏览器扩展管理
`browser-cli-inspect`	页面检查（DOM/screenshot）
`browser-cli-actions-input`	输入操作（click/type/scroll）
`browser-cli-actions-observe`	观察操作（wait/query）
`browser-cli-debug`	调试工具
`browser-cli-state`	状态管理（cookies/storage）

所有浏览器命令通过 `addGatewayClientOptions` 共享 Gateway RPC 连接选项。

33.2.9 `cron` — 定时任务管理

`cron` 命令管理基于 cron 表达式的定时任务调度。它是一个延迟加载的子命令，注册在 `cron-cli/register.ts` 中，通过 Gateway RPC 与运行中的 Gateway 通信来管理定时任务。

33.2.10 `nodes` — 节点管理

`nodes` 命令管理分布式部署中的节点。同样是延迟加载的子命令，注册在 `nodes-cli/register.ts` 中。节点管理涉及节点发现、节点状态查询和节点间通信。

33.2.11 `doctor` — 健康检查与诊断

`doctor` 是 OpenClaw 的“全科医生”——检测并修复常见问题：

```
openclaw doctor                  # 交互式诊断
openclaw doctor --repair         # 自动修复
openclaw doctor --fix            # --repair 的别名
openclaw doctor --force          # 激进修复（覆盖自定义配置）
openclaw doctor --non-interactive  # 非交互模式（仅安全迁移）
openclaw doctor --deep           # 扫描系统服务
```

`doctor` 命令注册在 `register.maintenance.ts` 中，与 `dashboard` 、 `reset` 、 `uninstall` 命令同属维护命令组：

命令	说明
`doctor`	健康检查 + 快速修复
`dashboard`	打开 Web 控制台
`reset`	重置本地配置/状态
`uninstall`	卸载 Gateway 服务 + 本地数据

`doctor` 的特殊之处在于——它被列入配置守卫的白名单（`ALLOWED_INVALID_COMMANDS`），即使配置文件无效也能跑，因为它本身就是用来修复配置问题的。

33.2.12 `security` — 安全审计

```
openclaw security audit           # 运行安全审计
openclaw security audit --deep    # 包含 Gateway 在线探测
openclaw security audit --fix     # 应用安全修复
openclaw security audit --json    # JSON 输出
```

`security audit` 的实现分为两步：

1. **修复**（可选）：`fixSecurityFootguns()` 收紧默认配置、修正文件权限（chmod）
2. **审计**：`runSecurityAudit()` 五层扫描（攻击面 → 配置 → 文件系统 → 通道 → 深度探测）

输出按严重程度分组（CRITICAL → WARN → INFO），每条发现包含 `checkId`、标题、详情和修复建议。

33.2.13 `onboard` — 引导向导

`onboard` 是新用户的入门向导，引导完成 Gateway 设置、工作空间创建和技能安装：

```
openclaw onboard                                # 交互式向导
openclaw onboard --flow quickstart              # 快速开始流程
openclaw onboard --non-interactive --accept-risk \
  --auth-choice token --token-provider anthropic \
  --token <key>                                 # 非交互自动化安装
```

`onboard` 命令拥有 CLI 中最多的选项（50+），涵盖了所有可能的配置组合：

- **认证选择**：支持 Anthropic、OpenAI、OpenRouter、Gemini、xAI、Moonshot 等 20+ 提供商
- **Gateway 配置**：端口、绑定模式（loopback/tailnet/lan/auto）、认证方式（token/password）
- **Tailscale 集成**：off / serve / funnel 三种模式
- **服务安装**：自动检测并安装为系统服务
- **通道/技能/健康检查**：均可通过 `--skip-*` 跳过

33.2.14 `update` — 版本更新

`update` 命令管理 OpenClaw 的版本升级，支持三种更新通道：

```
openclaw update                    # 使用当前通道更新
openclaw update --channel beta     # 切换到 Beta 通道
openclaw update --channel dev      # 切换到 Dev 通道（Git）
openclaw update status             # 查看更新状态
openclaw update wizard             # 交互式更新向导
openclaw --update                  # 简写形式
```

三种更新通道

通道	安装方式	来源
`stable`	npm（`latest` tag）	正式发布版本
`beta`	npm（`beta` tag）	预发布版本
`dev`	Git（main 分支）	源码编译

更新流程

`update` 命令实现了一套完整的更新编排流程：

1. **检测安装类型**：Git 源码 vs npm 全局包
2. **解析目标版本**：从 npm registry 获取 tag 对应的版本号
3. **降级确认**：如果目标版本低于当前版本，要求用户确认
4. **通道切换**：如果指定了新通道，写入配置文件
5. **执行更新**：
   - **Git 模式**：fetch → rebase → install deps → build → doctor
   - **npm 模式**：检测包管理器（npm/pnpm/bun）→ 全局安装
   - **通道切换**：`dev` → 克隆 Git 仓库；非 `dev` → 全局 npm 安装

6. **更新插件**：同步内置插件、更新 npm 安装的插件
7. **Shell 补全**：更新缓存、提示安装
8. **重启服务**：重启 Gateway 守护进程 + 运行 doctor

整个流程附带进度指示器（spinner），每个步骤显示耗时和成功/失败状态。更新完成后还会输出一条随机趣味语录（如 "The lobster has molted. Harder shell, sharper claws."）。

33.2.15 `backup` —— 本地备份

v2026.3.9 新增了 `openclaw backup` 命令组，用于本地数据的备份与校验（源码位于 `src/cli/program/register.backup.ts` 和 `src/infra/backup-create.ts`）。

```
# 创建完整备份（配置 + 工作区 + 技能）
openclaw backup create

# 仅备份配置文件（轻量级）
openclaw backup create --only-config

# 排除工作区目录
openclaw backup create --no-include-workspace

# 校验已有备份的完整性
openclaw backup verify <backup-path>
```

备份产物是一个包含 **Manifest 清单**的压缩包，Manifest 记录了 `schemaVersion`（格式版本）、`runtimeVersion`（OpenClaw 版本）以及每个资产（asset）的路径和校验信息。`verify` 命令会逐项核对 Manifest 与实际文件，报告缺失或损坏的条目。

本节小结

1. **`gateway`** 是最核心的命令，管理 WebSocket Gateway 的完整生命周期（前台运行/系统服务/探测/发现/RPC 调用），跨平台支持 launchd/systemd/schtasks
2. **`agent` / `message`** 分别处理 AI 对话和原始消息收发——`agent` 通过 Gateway 路由到 AI Agent，`message` 直接操作通道
3. **`channels` / `models`** 管理两大外部依赖——聊天通道和 AI 模型，其中 `models` 命令树深达三级，包含完整的认证、别名和回退链管理

4. **`doctor` / `security` / `onboard`** 构成运维三件套——诊断修复、安全审计、引导安装，`doctor` 被特殊豁免配置校验
5. **`update`** 实现了三通道（stable/beta/dev）更新编排，自动检测安装类型、管理包管理器、同步插件、重启服务
6. **所有命令统一采用** `runCommandWithRuntime(defaultRuntime, action)` 包装，确保错误处理一致；支持 `--json` 机器可读输出模式